

Web-Security und Intrusion Prevention

SoSe 26

Prof. R. Zahoransky

Hochschule Furtwangen (University of Applied Science)

Aufgabenblatt 1

Lernziele: Durchführen und Schützen vor SQL- und Command-Line Injection, Umgang mit Container

Abgabetermin wird bekannt gegeben

Aufgabe 1: Verstehen der Webanwendung

Entpacken Sie die Datei sql-injection.zip in einem Verzeichnis Ihrer Wahl. Darin finden Sie einige Python-Skripte, ein Dockerfile sowie eine docker-compose config.

Dockerfile.website

Die Datei Dockerfile.website erstellt ein Docker-Image, um eine Python Flask Webseite darzustellen. Machen Sie sich mit dem Inhalt vertraut:

- Es wird ein neues Verzeichnis erstellt, in welchem die Python-Skripte für die Webseite später eingebunden werden
- Es wird die Datei createSQLiteDatabase.py kopiert und während dem Bauen ausgeführt. Dieses Skript erzeugt eine Benutzerdatenbank für die Webseite.
- Flask (ein Python Framework für die Erstellung von Webseiten) wird mit pip installiert
- Die Dockerfile gibt an, dass der Container Port 5000 für die Kommunikation nach außen (zur Darstellung der Webseite) zur Verfügung stellt
- Zum Schluss wird mit python die Datei app.py gestartet

Docker-compose.yml

Diese Datei stellt die Konfiguration für docker compose bereit. Compose ist ein Tool, um mehrere Docker Container gleichzeitig zu starten, zu konfigurieren und in eigenen Netzwerken miteinander zu verbinden. Machen Sie sich auch mit diesem Inhalt vertraut, um zu sehen, welche Dateien Sie später bearbeiten können:

- Es wird ein Dienst mit dem Namen „website“ erstellt:
 - Dieser startet, indem aus Dockerfile.website der Docker-Container gebaut und anschließend gestartet wird.
 - Es wird Port 5000 für die Kommunikation mit dem Container geöffnet
 - Es werden die vier notwendigen Python-Skripte in den Container gemountet. Das bedeutet, dass lokale Änderungen an diesen Dateien auch im Container sichtbar sind
- Es wird ein Dienst mit dem Namen proxy erstellt:
 - Dieser Dienst benutzt ein fertiges Docker-Image mit dem Namen nginx aus dem Internet. Es ist ein Container, in welchem der Webserver NginX installiert ist. In dieser Aufgabe wird NginX als Reverse Proxy (Web Application Firewall oder WAF) verwendet.
 - Der Dienst wird erst gestartet, wenn der Dienst website läuft

- Die Konfigurationsdatei `nginx-filter.conf` wird in das Verzeichnis `/etc/nginx.conf` im Container gemountet. Dies ist die Konfigurationsdatei, welche von NginX beim Start geladen wird.

Nginx-filter.conf

In der Datei `nginx-filter.conf` sehen Sie die Konfiguration von NginX:

- NginX soll einen http-Server auf Port 8080 starten
- Wenn der Pfad „/“ aufgerufen wird, soll die Anfrage an <http://website:5000> weitergeleitet werden. Dieser Name ist direkt verwendbar, da `docker-compose` sich bei den Docker-Container als DNS-Server einträgt und die Namen auflöst. Die Namen entsprechen den Service-Definitionen aus der Datei `docker-compose.yml`.
- Vor der Weiterleitung haben Sie noch die Möglichkeit GET-Parameter zu filtern. Dies werden wir später benutzen.

Aufgabe 2: Starten und Stoppen der Container

- Laden und bauen Sie die notwendigen Docker-Images mit `docker compose build`.
- Starten Sie die erstellten Images mit dem Befehl `docker compose up`.

Steuerung der Container:

Sie sehen jetzt die Log-Ausgaben der Container. Mit STRG+C beenden Sie die laufenden Container. Sie können mit `d` zur Konsole zurückkehren, ohne die Container zu stoppen (oder Sie starten direkt mit `docker compose up -d`). Wenn Sie den Befehl `docker compose logs -f` eingeben, sehen Sie wieder die Ausgaben (Ausgabe mit STRG+C beenden).

Befehle im Container ausführen

Für diese Aufgabe ist es nicht notwendig, in die Container zu wechseln, Sie können es dennoch testen:

- Lassen Sie sich mit dem Befehl `docker ps` alle laufenden Container anzeigen.
- Mit dem Befehl `docker exec -it <Containername> bash` starten Sie eine neue Kommandozeile im Container, in die Sie automatisch hineinwechseln. Der Schalter `-it` gibt an, dass es sich um eine interaktive Sitzung handelt.
- Tippen Sie `exit` im Container ein, um zurückzukehren.

Die Containern und Images löschen

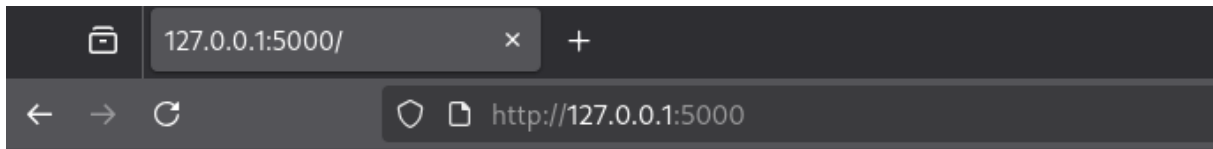
- Mit `docker system prune` können Sie (am Ende des Praktikums) alle erstellten Images und Container wieder löschen. Laufende Container und genutzte Images werden nicht gelöscht. Sie müssen laufenden Container vorher beenden.

Aufgabe 3:

Beim Start mit `docker compose up` sehen Sie den Hinweis, dass Python selbst einen Webserver startet. Er ist auf Port 5000 erreichbar (der von Docker Compose auch freigegeben wurde):

```
website-1 | WARNING: This is a development server. Do not use it in a production
website-1 | deployment. Use a production WSGI server instead.
website-1 | * Running on all addresses (0.0.0.0)
website-1 | * Running on http://127.0.0.1:5000
website-1 | * Running on http://172.18.0.2:5000
```

Öffnen Sie den Browser und navigieren Sie auf diesen Webserver:



Willkommen. Bitte einloggen:

Benutzername: Password:

Sollte der Zugriff nicht funktionieren, versuchen Sie die IP-Adresse des Docker-Container selbst.

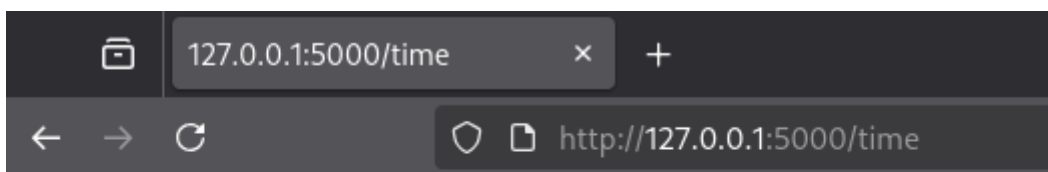
Sicherlich haben Sie sich die Python-Skripte angesehen, kennen also auch ein Benutzername und Passwort. Versuchen Sie sich einzuloggen.

Aufgabe 4 Greifen Sie die Webanwendung an

- a) Die Webseite hat (mindestens) zwei Sicherheitslücken. Versuchen Sie eine SQL-Injection.
- b) Versuchen Sie alle Zeilen der Datenbank anzeigen zu lassen.

Aufgabe 5 „Pw0nen“ Sie den Server mit einer Command-Line Injection

Navigieren Sie auf `127.0.0.1:5000/time`. Es ist ein Dienst, der Ihnen die aktuelle Systemzeit in benutzerdefinierter Formatierung anzeigt.



Darstellung der Systemzeit:

Wed Mar 11 15:24:46 UTC 2026

Lesen Sie sich den Text (und nochmals die Datei ShowDate.py) durch. Mit dem GET-Parameter `format` können Sie sich die Zeit in verschiedenen Formaten zurückgeben lassen.

Hierfür führt die Webseite den Command-Line Befehl `date` aus und hängt den `format`-Parameter an den Befehl an. Die Ausgabe wird dann in der Webseite angezeigt.

- a) Greifen Sie die Webseite an und lassen Sie sich den Inhalt des aktuellen Verzeichnisses anzeigen. Beantworten Sie hierfür auch folgende Frage: Wie können Sie mehrere Befehle hintereinander in der Konsole ausführen?
- b) Als welchen Benutzer haben Sie Zugriff auf das System?
- c) Wie ist der Inhalt der Datei `/etc/passwd`?

Aufgabe 6 Machen Sie die Anwendung (teilweise) sicherer

- a) Die WAF ist bereits aktiv (es ist der NginX Reverse Proxy), sie wurde nur noch nicht konfiguriert. Ändern Sie `docker-compose.yml` so, dass ein Zugriff nur noch über die WAF möglich ist.
- b) Verhindern Sie Ihre SQL-Injection, indem Sie in der Datei `nginx-filter.conf` nach einem (oder mehreren) Schlüsselwörter filtern. Um nach mehreren Schlüsselworten zu suchen, können Sie weitere `if`-Blöcke hinzufügen.

HINWEIS: Leider lädt NginX nicht automatisch neu. Daher müssen Sie die Container stoppen und mit `docker compose up` neustarten, damit die Änderungen wirksam werden.

- c) Die Anwendung benutzt GET-Parameter für den Login. Ist dies gut? Worin liegt das Problem? Ändern Sie die Anwendung so, dass Benutzername und Passwort als POST übertragen werden.
- d) Ändern Sie die SQL-Abfrage in `HandleLogin.py` zu einem Prepared Statement
- e) Können Sie auch die Command-Line-Injection Lücke in NginX oder im Code verhindern?
- f) Fallen Ihnen weitere Maßnahmen ein?